MODULE 5

# Data, Integration & Reliability

*Renaissance Developer Academy — Detailed Lesson Plan*

Week 5 of 10 | 5 Days | Full-Time Intensive
*Badge Earned: Reliable Systems Engineer*

Prerequisite: Module 4 — Systems Architect
DRAFT — March 2026

# Module Overview

Module 4 taught you to design systems on the whiteboard. Module 5 asks: **what happens when your design meets reality?** Databases corrupt, APIs timeout, networks partition, dependencies fail, and traffic spikes arrive at 3 AM. This week is about building systems that **survive** — and about developing the discipline to know, quantitatively, whether they're healthy.

The week covers three interconnected disciplines: **data integration** (getting data to flow reliably between services), **site reliability engineering** (defining what "healthy" means and measuring it), and **chaos engineering** (deliberately breaking things to build confidence in your system's resilience). Students apply these to their Module 3 full-stack application and their Module 4 spec-driven application, transforming working code into production-worthy systems.

**Connection to the JD:** This module maps directly to Site Reliability Engineering ("define reliability standards, drive post-incident improvements, design capacity planning"), Data Integration ("define data integration patterns, architect large-scale data flows"), and the Reliability responsibility ("establish SLOs/SLIs across teams, build resilient systems, drive reliability culture").

## Learning Objectives

1. Design and implement data integration patterns (ETL, event-driven, API gateway) with documented trade-offs for each pattern choice
2. Define SLOs (Service Level Objectives) and implement SLIs (Service Level Indicators) for a real service, including error budget calculations
3. Practice chaos engineering: introduce controlled failures, observe system behavior, and write post-incident reviews
4. Build monitoring and observability into applications: structured logging, metrics, alerting, and dashboards
5. Write incident response runbooks and conduct post-incident reviews following blameless retrospective practices
6. Integrate monitoring into CI/CD pipelines: automated alerts on deployment failures, test coverage regressions, and performance degradation

## Pre-Work (Assigned End of Module 4)

**Due before Day 1. Estimated time: 2–3 hours.**

1. **Reading:** Read Google's SRE Book Chapter 4 ("Service Level Objectives") — free at sre.google/sre-book. Upload to NotebookLM and generate an Audio Overview. Focus on: what is the relationship between SLOs, SLIs, and error budgets?
2. **Audit:** Review your Module 3 application. List every external dependency (database, APIs, cloud services). For each: what happens if it becomes unavailable for 5 minutes? For 1 hour? Do you have any monitoring in place? Be honest.

3. **Reading:** Read DDIA Chapter 4 (Encoding and Evolution) — focus on schema evolution and backward/forward compatibility. This connects to data integration patterns on Day 2.
4. **Tool:** Install a load testing tool: hey (Go), wrk, or k6. Verify it works: run hey -n 100 -c 10 http://localhost:3000/health against any local service.

## Week-at-a-Glance

| Day | Theme | Primary Tools | Key Output |
|---|---|---|---|
| **Day 1** | SRE Fundamentals: SLOs, SLIs & Error Budgets | Claude Code, monitoring tools | SLO definitions, SLI implementations, dashboard |
| **Day 2** | Data Integration Patterns & Pipeline Build | Kiro, Claude Code, GitHub Actions | Data pipeline connecting 2+ services |
| **Day 3** | Chaos Engineering: Breaking Things on Purpose | Chaos tools, monitoring, all AI tools | Chaos experiments, incident response runbook |
| **Day 4** | Observability Deep Dive & CI/CD Integration | Logging/metrics tools, GitHub Actions | Full observability stack, CI/CD monitoring alerts |
| **Day 5** | Incident Simulation, Reflection & Calibration | All tools, NotebookLM | Post-incident review, reliability ADR, dev map v5 |

# Day 1: SRE Fundamentals — SLOs, SLIs & Error Budgets

**Theme:** Define what "healthy" means quantitatively, measure it automatically, and understand the relationship between reliability and feature velocity

**Goal:** Every student has defined SLOs for their Module 3 application, implemented SLIs that measure them in real-time, calculated their error budget, and has a basic monitoring dashboard showing system health.

## Schedule

| 09:00 | **Week 5 Opening: Why Reliability Is a Feature** | 30 min |
|---|---|---|

- The shift: Module 4 asked "is this well-designed?" Module 5 asks "will this survive?"
- The Renaissance Developer's Reliability responsibility: "Establish SLOs/SLIs, build resilient systems, drive reliability culture"
- Reliability is not "keeping things running" — it is making a quantitative promise about service quality and keeping it
- Share pre-work dependency audits: most students realize they have zero monitoring and no idea what happens when dependencies fail
- This week's mantra: "If you can't measure it, you can't manage it."

| 09:30 | **SRE Fundamentals (Concept Session)** | 75 min |
|---|---|---|

- The SRE hierarchy: SLAs → SLOs → SLIs → Error Budgets
- — SLA (Service Level Agreement): the contractual promise to users (external, legal)
- — SLO (Service Level Objective): the internal target for service quality (stricter than SLA)
- — SLI (Service Level Indicator): the metric that measures whether you're meeting the SLO
- — Error Budget: the amount of unreliability you're allowed before you must prioritize reliability over features
- Choosing good SLIs (the art of measuring what matters):
- — Availability: percentage of successful requests (not server uptime — the user's experience matters)
- — Latency: response time distribution (p50, p95, p99 — averages lie)
- — Throughput: requests per second at acceptable latency
- — Correctness: percentage of responses that are accurate (crucial for data-intensive applications)
- — Freshness: how stale is the data? (important for dashboards, feeds, search results)
- Setting realistic SLOs:
- — 99.9% availability = ~8.7 hours of downtime per year = 43 minutes per month
- — 99.99% availability = ~52 minutes per year = 4.3 minutes per month

| | |
|---|---|
| | • — The cost of each additional 9 is exponential — what does your user actually need?<br>• Error budget math: if SLO is 99.9%, error budget is 0.1% of requests. Burn the budget = stop shipping features and fix reliability.<br>• Live example: define SLOs for a food delivery app (availability for ordering, latency for search, correctness for pricing) |

| | | |
|---|---|---|
| **10:45** | **Break** | 15 min |
| | • Questions about SLO concepts | |

| | | |
|---|---|---|
| **11:00** | **Hands-On: Define SLOs for Your Application** | 60 min |
| | • EXERCISE 1: Define SLOs for your Module 3 full-stack application<br>• For each of these SLI categories, define an SLO and justify it:<br>• — Availability SLO: "___% of requests will return non-5xx responses, measured over a 28-day window"<br>• — Latency SLO: "p95 response time will be under ___ms for API endpoints, measured over a 28-day window"<br>• — Correctness SLO (if applicable): "___% of data mutations will be accurately reflected within ___ms"<br>• For each SLO, calculate the error budget:<br>• — How many failed requests per month does your budget allow?<br>• — How many minutes of downtime per month?<br>• Write an ADR: "Why we chose these SLO targets" — context includes user expectations, team capacity, and system architecture<br>• Document in your design journal alongside your Module 4 architecture work<br>• Pair review: do your partner's SLOs seem realistic? Too aggressive? Too lenient? Why? | |

| | | |
|---|---|---|
| **12:00** | **Lunch** | 60 min |
| | • Optional: SLO calibration discussions | |

| | | |
|---|---|---|
| **13:00** | **Implementing SLIs: Metrics, Logging & Measurement** | 90 min |
| | • EXERCISE 2: Implement SLIs in your Module 3 application<br>• Add instrumentation that measures your SLOs in real-time:<br>• — Availability SLI: middleware that counts total requests vs. error responses (5xx)<br>• — Latency SLI: middleware that measures and records response time for every request<br>• — Implementation options (choose based on your stack):<br>• Option A: Prometheus-style metrics (prom-client for Node, prometheus_client for Python) — most standard<br>• Option B: Custom middleware logging to structured JSON — simplest to implement | |

| | |
|---|---|
| | • Option C: Application-level metrics with a hosted service (free tiers: Grafana Cloud, Datadog, New Relic)<br>• Add a /metrics endpoint that exposes:<br>• — Total request count (by endpoint, by status code)<br>• — Request duration histogram (buckets: 10ms, 50ms, 100ms, 250ms, 500ms, 1000ms, 5000ms)<br>• — Error count (by type: 4xx client errors, 5xx server errors)<br>• — Uptime since last restart<br>• Use Claude Code CLI for implementation — apply verification: are metrics accurate? Is there overhead?<br>• Test: generate traffic with your load testing tool, verify metrics reflect actual behavior |

| 14:30 | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| 14:45 | **Building an SLO Dashboard** | 60 min |
|---|---|---|
| | • EXERCISE 3: Create a monitoring dashboard for your SLOs<br>• — Option A (recommended for learning): Build a simple dashboard page in your app that reads /metrics and displays:<br>• — Current availability (last 1 hour, last 24 hours, last 7 days)<br>• — Latency percentiles (p50, p95, p99 for the last hour)<br>• — Error budget remaining this month (visual indicator: green/yellow/red)<br>• — Request rate trend (requests per minute over the last hour)<br>• — Option B (production-realistic): Connect to Grafana Cloud (free tier) and build a Grafana dashboard<br>• — Option C (minimal): CLI-based dashboard that queries /metrics and prints a summary<br>• Regardless of option, your dashboard must answer: “Are we meeting our SLOs right now?” in under 5 seconds<br>• Push dashboard code and deploy — this becomes a permanent part of your application | |

| 15:45 | **Alerting Basics** | 30 min |
|---|---|---|
| | • EXERCISE 4: Define alert rules based on your SLOs<br>• Write alert definitions (even if you don’t fully automate them yet):<br>• — Critical Alert: error rate exceeds 5% of requests in a 5-minute window (availability SLO threatened)<br>• — Warning Alert: p95 latency exceeds SLO target for 10 consecutive minutes<br>• — Info Alert: error budget consumed past 50% threshold for the month<br>• — For each: who gets notified? What’s the expected response? What’s the escalation path?<br>• Document alert definitions in a Markdown file: alerts.md in your repo | |

| | |
|---|---|
| | • (Advanced): Implement one alert as a GitHub Actions scheduled workflow that checks /metrics every 5 minutes |

| 16:15 | **Day 1 Wrap-Up** | 30 min |
|---|---|---|
| | • End-of-day state: SLOs defined and documented, SLIs implemented with /metrics endpoint, dashboard showing real data, alert rules written<br>• HOMEWORK: Run your load testing tool against your deployed application for 10 minutes. Capture the metrics. Do you meet your SLOs under load?<br>• HOMEWORK: Read about ETL vs. ELT vs. event-driven integration patterns (article provided)<br>• Preview: tomorrow we build data pipelines and tackle integration patterns | |

***Instructor Notes — Day 1:*** *Day 1's SRE content is where many mid-level engineers encounter formal reliability engineering for the first time. The most common mistake is setting SLOs that are either impossibly strict (99.999%) or meaninglessly loose (95%). Push students to justify their targets with user expectations and team capacity. The error budget concept is often a lightbulb moment — the idea that reliability and feature velocity are in direct tension, managed through a quantitative budget. The /metrics implementation should be done with AI tools but verified carefully — incorrect metrics are worse than no metrics because they create false confidence. The dashboard exercise gives tangible, visual satisfaction after a concept-heavy morning.*

# Day 2: Data Integration Patterns & Pipeline Build

**Theme:** Design and implement data flows between services using the right integration pattern for each situation

**Goal:** Every student has built a data pipeline connecting at least 2 services, has documented their integration pattern choice with a trade-off analysis, and understands when to use ETL, event-driven, or API-based integration.

## Schedule

| 09:00 | **Data Integration Patterns (Concept Session)** | 60 min |
|---|---|---|
| | • The three fundamental integration patterns:<br>• — Pattern 1 — API-Based (Synchronous): Service A calls Service B's API directly<br>• Pros: simple, immediate response, easy to reason about<br>• Cons: tight coupling, cascading failures, blocked if B is down<br>• Use when: low latency required, small data volumes, strong consistency needed<br>• — Pattern 2 — Event-Driven (Asynchronous): Service A publishes events, Service B subscribes<br>• Pros: loose coupling, resilient to failures, natural scaling<br>• Cons: eventual consistency, harder to debug, event ordering challenges<br>• Use when: high throughput, services can tolerate delay, many consumers for same data<br>• — Pattern 3 — ETL/ELT (Batch): Extract data from source, transform, load into destination on a schedule<br>• Pros: handles large volumes, source system not impacted, data can be reshaped freely<br>• Cons: data is always stale (by schedule interval), complex transformations, monitoring overhead<br>• Use when: analytics, reporting, data warehousing, legacy system integration<br>• The API Gateway pattern: a single entry point routing requests to appropriate backend services<br>• — Handles: authentication, rate limiting, request routing, response aggregation<br>• — Trade-off: reduces client complexity but introduces a single point of failure<br>• Schema evolution and compatibility (from DDIA pre-reading):<br>• — Forward compatibility: new code can read old data<br>• — Backward compatibility: old code can read new data<br>• — Why this matters: in any integration, producer and consumer may evolve at different speeds | |

| 10:00 | **Integration Design Workshop** | 60 min |
|---|---|---|
| | • EXERCISE 5: Design a data integration architecture<br>• Scenario: you are building a dashboard that aggregates data from 3 sources:<br>• — Source 1: Your Module 3 application's database (user activity data) | |

| | |
|---|---|
| | • — Source 2: A third-party API (weather, stock prices, news, or any public API you choose)<br>• — Source 3: A file-based data source (CSV or JSON file updated periodically)<br>• Design decisions to make and document:<br>• — Which integration pattern for each source? (API, event-driven, ETL) — and why?<br>• — Where does data transformation happen? (at source, in transit, at destination)<br>• — How do you handle source failures? (retry, fallback, circuit breaker)<br>• — What is the freshness requirement for each data source?<br>• — How do you handle schema changes in the third-party API?<br>• Write a Kiro-style spec: requirements.md for the integration, design.md for the architecture<br>• Use the trade-off analysis matrix from Module 4 to compare integration pattern options |

| **11:00** | **Break** | 15 min |
|---|---|---|
| | • Questions about pattern selection | |

| **11:15** | **Pipeline Build Sprint: Phase 1** | 75 min |
|---|---|---|
| | • EXERCISE 6: Build the data pipeline<br>• Implement your integration design — at minimum, connect 2 of the 3 sources:<br>• — API integration: fetch data from the third-party API with proper error handling:<br>• — Retry logic with exponential backoff<br>• — Circuit breaker: stop calling after N consecutive failures, try again after cooldown<br>• — Timeout handling: don't block forever waiting for a response<br>• — Rate limiting: respect the API's rate limits<br>• — File/batch integration: read, transform, and load data from the file source:<br>• — Schema validation: verify the data matches expected format before processing<br>• — Error handling: skip bad records, log them, don't crash the pipeline<br>• — Idempotency: running the pipeline twice with the same data produces the same result<br>• — Database integration: query your Module 3 app's data for the dashboard<br>• Use Claude Code for implementation but apply the Verification Lens — integration code is where edge cases live<br>• Commit incrementally: each integration source is a separate, testable module | |

| **12:30** | **Lunch** | 60 min |
|---|---|---|
| | • Optional: API debugging office hours | |

| **13:30** | **Pipeline Build Sprint: Phase 2** | 90 min |
|---|---|---|

| | |
|---|---|
| | • EXERCISE 6 (continued): Complete and connect the pipeline<br>• — Build the aggregation layer: combine data from all sources into a unified view<br>• — Build a simple dashboard endpoint or page that displays the aggregated data<br>• — Add SLIs to the pipeline: track success rate, processing time, data freshness per source<br>• — Add monitoring to your /metrics endpoint: pipeline_runs_total, pipeline_errors_total, pipeline_latency<br>• EXERCISE 7: Write integration tests for the pipeline<br>• — Test with mocked external sources: what happens when the API returns 500? When the file is malformed?<br>• — Test the circuit breaker: after 5 failures, does it stop trying? After cooldown, does it retry?<br>• — Test idempotency: run the batch pipeline twice, verify no duplicate data<br>• Add pipeline tests to your CI/CD pipeline in GitHub Actions |

| 15:00 | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| 15:15 | **Integration Pattern ADR & Documentation** | 45 min |
|---|---|---|
| | • EXERCISE 8: Write ADRs for your integration pattern choices<br>• — ADR 1: Why you chose [pattern] for the API source (vs. alternatives)<br>• — ADR 2: Your error handling strategy (circuit breaker vs. simple retry vs. dead letter queue)<br>• — Include: the trade-off matrix scores for each option considered<br>• Update your architecture document with a data flow diagram showing all integrations<br>• Commit all work: pipeline code, tests, ADRs, updated architecture document | |

| 16:00 | **Day 2 Wrap-Up** | 30 min |
|---|---|---|
| | • Accountability partner check: demo your pipeline — show data flowing from sources to dashboard<br>• HOMEWORK: Deploy the pipeline alongside your application. Verify it works in staging.<br>• HOMEWORK: Read about chaos engineering principles (Principles of Chaos Engineering — principlesofchaos.org)<br>• Preview: tomorrow we break things on purpose | |

***Instructor Notes — Day 2:*** *Day 2's data integration content bridges the gap between Module 4's theoretical distributed systems and real implementation. The pipeline build is deliberately scoped to be achievable in one day — 2 sources minimum, 3 if time allows. The most common struggle is error handling: students build the happy path quickly with AI tools but underestimate the complexity of retries, circuit breakers, and idempotency. These are exactly the areas where Module 2's Bug Log categories (logic errors, performance anti-patterns) manifest in practice. Push students to write integration tests BEFORE they consider the pipeline done — untested*

*integration code is a reliability liability.*

# Day 3: Chaos Engineering — Breaking Things on Purpose

**Theme:** Build confidence in your system's resilience by deliberately introducing failures and observing how the system responds

**Goal:** Every student has conducted at least 3 chaos experiments on their application, has documented observed failures and unexpected behaviors, and has written an incident response runbook.

## Schedule

| 09:00 | **Chaos Engineering Principles (Concept Session)** | 45 min |
|---|---|---|
| | • What chaos engineering IS and IS NOT:<br>• — IS: a disciplined practice of introducing controlled failures to discover system weaknesses<br>• — IS NOT: randomly breaking things in production without a hypothesis<br>• The Chaos Engineering experiment cycle:<br>• — Step 1: Define steady state — what does "normal" look like? (your SLOs from Day 1!)<br>• — Step 2: Hypothesize — "We believe the system will [expected behavior] when [failure condition]"<br>• — Step 3: Introduce the failure — in a controlled way, with a way to stop immediately<br>• — Step 4: Observe — watch metrics, logs, user experience. Did reality match the hypothesis?<br>• — Step 5: Learn — if the hypothesis was wrong, that's a finding. Document it. Fix it.<br>• Types of failures to test:<br>• — Dependency failure: database unavailable, external API down, message queue full<br>• — Resource exhaustion: CPU maxed, memory full, disk full, connection pool exhausted<br>• — Network degradation: high latency, packet loss, DNS failure, timeout<br>• — Data corruption: invalid data in the database, malformed API responses<br>• — Load spike: 10x normal traffic suddenly (the "bursting" scenario from Module 4 Problem 1)<br>• Connection to the JD's Systems Thinking behavior: "conducts chaos engineering experiments"<br>• Connection to the Reliability responsibility: "lead reliability initiatives for your area" | |

| 09:45 | **Chaos Experiment Design Workshop** | 45 min |
|---|---|---|
| | • EXERCISE 9: Design 5 chaos experiments for your application<br>• For each experiment, write a hypothesis card: | |

| | |
|---|---|
| | • — Experiment name<br>• — Hypothesis: “We believe [system behavior] when [failure condition]”<br>• — Blast radius: what’s affected? Can we limit the damage?<br>• — Steady state metrics: which SLIs will we watch?<br>• — Abort condition: when do we stop the experiment?<br>• — Method: how exactly will we introduce the failure?<br>• Suggested experiments (adapt to your application):<br>• — Experiment 1: Kill the database — docker stop your database container while the app is running<br>• — Experiment 2: Slow the API — add artificial latency (300ms–1000ms) to an external API call<br>• — Experiment 3: Spike traffic — 10x load test while the application is serving normal requests<br>• — Experiment 4: Corrupt data — insert invalid data into the database, observe how the app handles it<br>• — Experiment 5: Fill the connection pool — open maximum concurrent connections, observe degradation<br>• Peer review: exchange experiment designs with your accountability partner — are hypotheses testable? Are abort conditions clear? |

| 10:30 | **Break** | 15 min |
|---|---|---|
| | • Final experiment prep | |

| 10:45 | **Chaos Experiments: Execution Round 1** | 90 min |
|---|---|---|
| | • EXERCISE 10: Execute your first 3 chaos experiments<br>• Run against your LOCAL environment (not production — not yet):<br>• — For each experiment:<br>• 1. Establish steady state: verify SLIs are within SLO, app is functioning normally<br>• 2. Start recording: keep your dashboard visible, have log tailing running<br>• 3. Introduce the failure: execute the chaos action<br>• 4. Observe for 2–5 minutes: watch metrics, check user experience, note anomalies<br>• 5. Remove the failure: restore normal operation<br>• 6. Document findings immediately (memory fades fast):<br>• — Did reality match your hypothesis? If not, what happened instead?<br>• — Which SLIs were affected and by how much?<br>• — Did the application recover automatically? How long did recovery take?<br>• — Were there any cascading failures you didn’t expect?<br>• — What would happen if this occurred in production at peak traffic?<br>• Common finding: the database kill experiment reveals that most applications have no graceful degradation — they crash hard<br>• Common finding: the traffic spike reveals N+1 queries and connection pool limits that were invisible at normal load | |

| 12:15 | Lunch | 60 min |
|---|---|---|
| | • Informal: share the most surprising failure you observed | |

| 13:15 | Chaos Experiments: Execution Round 2 + Fix Sprint | 90 min |
|---|---|---|
| | • EXERCISE 10 (continued): Execute remaining 2 experiments<br>• EXERCISE 11: Fix the most critical finding from Round 1<br>• Triage your findings by severity:<br>• — Critical: system crashes or data loss on failure — fix this today<br>• — High: system degrades significantly, no automatic recovery — fix or document mitigation<br>• — Medium: system handles the failure but recovery is slow — improve when time allows<br>• — Low: minor degradation, system recovers quickly — acceptable risk, document<br>• Fix the highest-severity finding:<br>• — Implement the fix using Claude Code CLI<br>• — Apply your AI Code Review Checklist (reliability fixes are especially sensitive to AI quality)<br>• — Re-run the chaos experiment to verify the fix works<br>• — Commit with a message linking to the chaos experiment finding<br>• Pair exercise: run one of your partner's chaos experiments against their application. Do they get the same results? (Peer testing catches things self-testing misses) | |

| 14:45 | Break | 15 min |
|---|---|---|
| | • Stretch | |

| 15:00 | Incident Response Runbook Workshop | 60 min |
|---|---|---|
| | • EXERCISE 12: Write an incident response runbook for your application<br>• A runbook is the "what to do when things go wrong" document:<br>• Section 1 — Service Overview:<br>• — What the service does, who depends on it, architecture diagram reference<br>• — Key contacts: who owns this service? Who is the backup?<br>• Section 2 — Common Incidents (based on your chaos experiments):<br>• — For each known failure mode: symptoms, diagnosis steps, remediation steps, verification steps<br>• — Example: "Database unavailable" → check DB container status → restart if crashed → verify connectivity → check for data corruption<br>• Section 3 — Escalation:<br>• — When to escalate (e.g., error budget burned past 75%, data loss detected)<br>• — How to escalate (who to contact, what information to provide)<br>• Section 4 — Post-Incident Process:<br>• — Timeline construction, root cause analysis, action items, blameless retrospective format | |

| | |
|---|---|
| | • Write as a Markdown file in your repo: runbook.md<br>• Connection: the JD requires “drive post-incident improvements systematically” |

| 16:00 | Day 3 Wrap-Up | 30 min |
|---|---|---|
| | • Chaos Gallery Walk: post your most surprising experiment finding on the class board<br>• Cohort patterns: what types of failures are most common across everyone’s applications?<br>• HOMEWORK: Complete any remaining chaos experiment documentation. Deploy fixes to staging.<br>• Preview: tomorrow — observability deep dive and wiring monitoring into your CI/CD pipeline | |

***Instructor Notes — Day 3:*** *Day 3 is the most viscerally engaging day of Module 5 — students are literally breaking their own applications and watching them fail. The chaos experiments must be run locally first; if students try to break their production deployments on Day 3, intervene immediately. The database kill experiment (docker stop) is almost always the most revealing because most applications have zero graceful degradation for database failures. Use the Chaos Gallery Walk at end-of-day to create a cohort-wide “failure knowledge base” — everyone learns from everyone’s experiments. The incident response runbook exercise may feel bureaucratic to some students; frame it as the Renaissance Developer’s Reliability responsibility: “you’re not writing this for yourself, you’re writing it for the person who gets paged at 3 AM.”*

# Day 4: Observability Deep Dive & CI/CD Integration

**Theme:** Build a comprehensive observability stack and integrate reliability monitoring into your CI/CD pipeline

**Goal:** Every student has structured logging, metrics, and alerting integrated into their application and CI/CD pipeline, with automated checks that prevent deploying code that would degrade reliability.

## Schedule

| 09:00 | **The Three Pillars of Observability (Concept Session)** | 45 min |
|---|---|---|
| | • Observability = can you understand the internal state of your system from its external outputs?<br>• Pillar 1 — Logs: structured records of discrete events<br>• — Structured logging (JSON) vs. unstructured logging (text) — structured is searchable, analyzable<br>• — Log levels: ERROR (broken), WARN (degraded), INFO (business events), DEBUG (development only)<br>• — Correlation IDs: a unique request ID threaded through every log entry, enabling full request tracing<br>• — What NOT to log: passwords, tokens, PII, full request bodies with sensitive data<br>• Pillar 2 — Metrics: numerical measurements over time<br>• — Counter: monotonically increasing (requests, errors, processed items)<br>• — Gauge: current value (active connections, queue depth, memory usage)<br>• — Histogram: distribution of values (response time, payload size)<br>• — Your SLIs from Day 1 are metrics — now we make them comprehensive<br>• Pillar 3 — Traces: the path of a single request through multiple services<br>• — Distributed tracing: follow a request from frontend → API → database → external service<br>• — Span: a single operation within a trace (DB query, API call, cache lookup)<br>• — When to invest in tracing: when you have 3+ services and need to diagnose cross-service latency<br>• The observability mindset: don't just monitor what you expect to fail — instrument enough to diagnose surprises | |

| 09:45 | **Structured Logging Implementation** | 75 min |
|---|---|---|
| | • EXERCISE 13: Add comprehensive structured logging to your application<br>• — Replace any console.log/print statements with structured JSON logging<br>• — Add a logging middleware that automatically logs for every request:<br>• — Request: method, path, user ID (if authenticated), correlation ID, timestamp<br>• — Response: status code, duration_ms, correlation ID<br>• — Add business event logging for key user actions: | |

| | |
|---|---|
| | • — User created, user logged in, item created, payment processed, etc.<br>• — Use INFO level with consistent event naming (e.g., “event”: “user.created”)<br>• — Add error context logging: when an error occurs, log the full context (request, user, stack trace) but scrub sensitive data<br>• — Implement correlation ID: generate a unique ID for each request, pass it through all layers<br>• Use Claude Code for implementation — verify: is sensitive data scrubbed? Are log levels appropriate?<br>• Test: trigger various actions, read the logs, verify you could diagnose a problem from logs alone |

| **11:00** | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| **11:15** | **Metrics Enhancement & Alerting Implementation** | 75 min |
|---|---|---|
| | • EXERCISE 14: Expand metrics and implement at least one automated alert<br>• Enhance your Day 1 /metrics endpoint with additional indicators:<br>• — Pipeline metrics (from Day 2): pipeline_runs_total, pipeline_errors_total, source_freshness_seconds<br>• — Business metrics: active_users_total, items_created_total, api_calls_by_endpoint<br>• — Infrastructure metrics: db_connection_pool_active, db_query_duration, memory_usage_bytes<br>• Implement at least one automated alert:<br>• — Option A: GitHub Actions scheduled workflow that checks /health every 5 min, creates an issue if unhealthy<br>• — Option B: In-app alert that writes to a Slack/Discord webhook when error rate spikes<br>• — Option C: Email alert via a free service (e.g., Uptime Robot) that monitors your deployment URL<br>• Test the alert: deliberately trigger the failure condition, verify the alert fires<br>• Document: which alert did you implement? What failure triggers it? What’s the expected response? | |

| **12:30** | **Lunch** | 60 min |
|---|---|---|
| | • Optional: monitoring tool comparisons | |

| **13:30** | **CI/CD Reliability Integration** | 90 min |
|---|---|---|
| | • EXERCISE 15: Add reliability checks to your GitHub Actions pipeline<br>• Your CI/CD pipeline should now catch reliability regressions before they reach production:<br>• — Performance regression check: run a quick load test (50 requests, 10 concurrent) in CI, fail if p95 > threshold | |

| | |
|---|---|
| | • — Post-deploy health check: after deploying to staging, verify /health returns 200 with all dependencies healthy<br>• — Test coverage gate: maintain >80% coverage (from Module 3), block merge if it drops<br>• — Security scan: npm audit / pip-audit as a CI step (from Module 3, ensure it's still active)<br>• — Integration test gate: pipeline tests from Day 2 must pass before merge<br>• — (Advanced) Canary deployment: deploy to a small percentage of traffic first, verify SLIs, then full rollout<br>• — (Advanced) Rollback automation: if post-deploy health check fails, automatically revert to previous version<br>• Update your pipeline to include deployment notification: a GitHub Actions step that posts to Slack/Discord when deployment succeeds or fails<br>• Goal: your pipeline catches the problems that your chaos experiments found BEFORE they reach production |

| **15:00** | **Break** | 15 min |
|---|---|---|
| | • Stretch | |

| **15:15** | **Reliability Architecture Documentation** | 45 min |
|---|---|---|
| | • EXERCISE 16: Update your architecture document with reliability architecture<br>• Add a new section to your architecture doc:<br>• — SLOs and SLIs: the targets and how they're measured<br>• — Monitoring architecture: what's instrumented, where metrics flow, how alerts fire<br>• — Data integration map: data flow diagram with integration patterns labeled<br>• — Known failure modes: from chaos experiments, with severity and mitigation<br>• — Incident response: link to runbook.md<br>• — CI/CD reliability gates: what automated checks prevent degradation<br>• Write a Reliability ADR: "Why we chose these SLO targets and this monitoring approach" | |

| **16:00** | **Day 4 Wrap-Up** | 30 min |
|---|---|---|
| | • End-of-day state: structured logging, enhanced metrics, at least 1 automated alert, CI/CD reliability gates, updated architecture doc<br>• HOMEWORK: Ensure everything is deployed and working. Run one more chaos experiment against staging to verify your fixes hold.<br>• Preview: tomorrow — the ultimate test: a simulated production incident | |

***Instructor Notes — Day 4:*** *Day 4 is implementation-heavy and ties together all of Module 5's themes. The structured logging exercise often reveals that students' applications are logging either nothing useful or everything (including sensitive data). Coach toward the balance: enough to diagnose problems, not so much that logs become noise. The CI/CD reliability integration is the day's most impactful exercise — it makes reliability improvements permanent by embedding*

*them in the deployment pipeline. Students who implement the performance regression check in CI often discover that their application is slower in CI than locally due to resource constraints — a good lesson about environment-specific behavior. The advanced exercises (canary deployment, rollback automation) are stretch goals; don't let students feel they've failed if they don't reach them.*

# Day 5: Incident Simulation, Reflection & Calibration

**Theme:** Put everything to the test in a simulated production incident, then reflect on the journey from builder to reliability engineer

**Goal:** Every student has participated in an incident simulation, written a post-incident review, and updated their Personal Development Map to reflect growth in reliability engineering and systems thinking.

## Schedule

| | | |
|---|---|---|
| **09:00** | **The Incident Simulation (Whole-Cohort Exercise)** | 120 min |
| | • EXERCISE 17: Live Incident Simulation<br>• The instructor introduces a series of cascading failures into a shared reference application (or a volunteer student's application):<br>• — 09:00 — Scenario briefing: you are the on-call engineer. The application is in production. Users are reporting issues.<br>• — 09:05 — Failure 1: Database performance degradation (slow queries — a missing index under load)<br>• — 09:20 — Failure 2: External API starts returning intermittent 500 errors (circuit breaker tested)<br>• — 09:40 — Failure 3: Memory leak causes gradual performance degradation (p95 latency climbs over 30 min)<br>• — 10:00 — Failure 4: Traffic spike (10x) coincides with the degraded system state (cascading failure)<br>• Students work in pairs as "incident response teams":<br>• — Use your monitoring dashboards, logs, and runbooks to diagnose each failure<br>• — Communicate findings to the class ("mock incident channel" — Slack or live verbal updates)<br>• — Propose and implement fixes in real-time (or propose mitigations if fix isn't immediate)<br>• — Track timeline: when did you detect each issue? How long to diagnose? How long to mitigate?<br>• 10:20 — Debrief begins: instructor reveals the full failure chain<br>• — Which teams detected issues fastest? What tools/metrics helped most?<br>• — Which failures went undetected? Why? What instrumentation was missing?<br>• — How did cascading failures make diagnosis harder?<br>• Key lesson: incidents are not individual failures — they are failure chains where each problem makes the next one harder to diagnose | |

| | | |
|---|---|---|
| **11:00** | **Break** | 15 min |
| | • Decompress — incident simulations are stressful | |

| | | |
|---|---|---|
| **11:15** | **Post-Incident Review Workshop** | 60 min |

| | |
|---|---|
| | • EXERCISE 18: Write a post-incident review for the simulated incident<br>• Use the blameless retrospective format:<br>• — Summary: what happened, who was affected, duration of impact<br>• — Timeline: chronological account of events, detection, diagnosis, and mitigation (from your notes)<br>• — Root cause analysis: the chain of failures and contributing factors (NOT “who made a mistake”)<br>• — Impact: which SLOs were violated? By how much? What was the error budget impact?<br>• — Action items: specific, assigned, time-bound improvements to prevent recurrence<br>• — Example: “Add missing index to users table — assigned: [name] — due: EOD Tuesday”<br>• — Example: “Implement circuit breaker for external API — assigned: [name] — due: next sprint”<br>• — Lessons learned: what would we do differently? What monitoring gaps did we discover?<br>• The “blameless” principle: focus on the system conditions that allowed the failure, not the people involved<br>• — Bad: “Engineer X forgot to add an index”<br>• — Good: “Our schema review process doesn’t include query plan analysis for high-traffic tables”<br>• Commit the post-incident review to your design journal |

| 12:15 | **Lunch** | 60 min |
|---|---|---|
| | • Informal incident war stories | |

| 13:15 | **Week 5 Retrospective & Development Map Update** | 60 min |
|---|---|---|
| | • Sailboat retrospective:<br>• — Wind: What worked? (often: SLO dashboards providing real confidence, chaos experiments revealing blind spots)<br>• — Anchor: What was hard? (often: implementing good observability, writing runbooks that are actually useful)<br>• — Rocks: Concerns for Modules 6–10?<br>• — Island: What are you most excited about?<br>• EXERCISE 19: Personal Development Map v5<br>• — Re-rate on five mindsets: how has Systems Thinking deepened? How about Outcome orientation (SLOs are outcome measurement)?<br>• — Competency layers: Module 5 targeted Layer 2 (Architect — SRE, Service Mgmt) and previewed Layer 3 (Enabler — driving reliability culture)<br>• — Milestone: you’re halfway through the program. You can build fast (M3), design well (M4), and make it reliable (M5).<br>• — Refined intention: what do you need most from Modules 6–10?<br>• Instructor 1:1s through the afternoon | |

| 14:15 | **Impossible Stuff Day: Self-Directed Exploration** | 105 min |
|---|---|---|
| | • Week 5 Impossible Stuff Day suggestions:<br>• — Implement advanced chaos: use a proper chaos tool (Toxiproxy, Chaos Monkey for Spring Boot, Pumba for Docker) instead of manual failures<br>• — Build a synthetic monitoring system: a cron job that tests your production app's critical paths every 5 minutes and reports SLI data<br>• — Explore distributed tracing: add OpenTelemetry to your application (jaeger or zipkin for visualization)<br>• — Read ahead: skim the ClawSwarm repository for Module 6's multi-agent orchestration week<br>• — Read ahead: "The Lean Startup" Chapters 1–3 for Module 7's business immersion week<br>• — Write a blog post: "What I learned by breaking my own application"<br>• — Build a "SLO calculator" tool: input your target availability, output the allowed downtime per day/week/month/year<br>• Share explorations in last 15 min | |

| 16:00 | **Badge Ceremony & Week 5 Close** | 30 min |
|---|---|---|
| | • Badge ceremony: Reliable Systems Engineer badge for students who completed all deliverables<br>• Achievements: best SLO dashboard, most revealing chaos experiment, best post-incident review, most comprehensive runbook<br>• Read the Developer Manifesto together (recurring ritual)<br>• Halfway mark celebration: you've completed 5 of 10 modules. You can now build, verify, architect, and ensure reliability.<br>• The second half of the program shifts: from individual capability to team impact (M6–7) to organizational influence (M8) to capstone integration (M9–10).<br>• Module 6 preview: Rapid Prototyping & Design Sprints — 48-hour hackathon, ClawSwarm multi-agent systems, user validation | |

***Instructor Notes — Day 5:*** *The incident simulation is Module 5's capstone and one of the program's most memorable experiences. Preparation is critical: pre-plan the failure chain, have a reference application ready (or coordinate with a volunteer student), and test the failures yourself beforehand. The simulation works best when failures cascade — each new failure should interact with previous ones (e.g., slow database + traffic spike = cascading timeout). Time the debrief carefully: students need to hear the "answers" while their experience is fresh. The post-incident review format should be modeled on Google's SRE book standards. The blameless principle is important culturally — if the simulation environment feels punitive, students will be afraid to admit what they missed. The halfway celebration should feel earned — acknowledge how far students have come since Day 1 of Module 1.*

# Appendix A: Exercise Summary

| # | Day | Exercise | Type |
|---|---|---|---|
| 1 | 1 | Define SLOs for Module 3 app: availability, latency, correctness + error budget calculation | Framework |
| 2 | 1 | Implement SLIs: /metrics endpoint with request counts, latency histograms, error rates | Build |
| 3 | 1 | Build SLO dashboard: real-time availability, latency percentiles, error budget status | Build |
| 4 | 1 | Define alert rules based on SLOs: critical, warning, info thresholds | Framework |
| 5 | 2 | Design data integration architecture for 3-source dashboard (pattern selection + trade-off analysis) | Design |
| 6 | 2 | Build data pipeline: API integration (circuit breaker, retry), batch processing (idempotent), DB query | Build |
| 7 | 2 | Write integration tests: mocked failures, circuit breaker behavior, idempotency verification | Testing |
| 8 | 2 | Write ADRs for integration pattern choices with trade-off matrix scores | Documentation |
| 9 | 3 | Design 5 chaos experiments with hypothesis cards (name, hypothesis, blast radius, abort condition) | Design |
| 10 | 3 | Execute 5 chaos experiments: document findings, SLI impact, unexpected behaviors | Experiment |
| 11 | 3 | Fix the most critical finding from chaos experiments, verify fix with re-run | Build |
| 12 | 3 | Write incident response runbook: service overview, common incidents, escalation, post-incident | Documentation |
| 13 | 4 | Implement structured JSON logging with correlation IDs, appropriate levels, sensitive data scrubbing | Build |
| 14 | 4 | Expand metrics, implement at least 1 automated alert (GitHub Actions, webhook, or uptime monitor) | Build |
| 15 | 4 | Add CI/CD reliability gates: performance regression, post-deploy health check, coverage + security | DevOps |
| 16 | 4 | Update architecture document with reliability section: SLOs, monitoring, failure modes, CI/CD gates | Documentation |
| 17 | 5 | Live incident simulation: cascading failures, real-time diagnosis, communication, timeline tracking | Simulation |
| 18 | 5 | Write blameless post-incident review: timeline, root cause, impact, action items, lessons learned | Documentation |
| 19 | 5 | Personal Development Map v5: halfway reflection, | Reflection |

| | | | |
|---|---|---|---|
| | | reliability growth evidence | |

# Appendix B: Deliverables Checklist

Complete all items to earn the Reliable Systems Engineer badge:

1. **SLO Definitions + ADR** — Documented SLOs (availability, latency, correctness) with error budget calculations and an ADR justifying target selection
2. **SLI Implementation + Dashboard** — /metrics endpoint exposing request counts, latency histograms, error rates; dashboard showing real-time SLO status with error budget burn indicator
3. **Data Pipeline** — Working pipeline connecting 2+ data sources with documented integration patterns, circuit breaker, retry logic, idempotency, and passing integration tests
4. **Chaos Experiment Documentation** — 5 chaos experiments with hypothesis cards, execution results, SLI impact measurements, findings, and at least 1 critical fix verified by re-run
5. **Incident Response Runbook** — runbook.md with service overview, common incidents (from chaos experiments), diagnosis and remediation steps, escalation paths, and post-incident process
6. **Observability Stack** — Structured JSON logging with correlation IDs, expanded metrics, at least 1 automated alert implemented and tested
7. **CI/CD Reliability Gates** — GitHub Actions pipeline with performance regression check, post-deploy health verification, test coverage and security scanning gates
8. **Post-Incident Review** — Blameless post-incident review of the Day 5 simulation: timeline, root cause chain, SLO impact, action items, lessons learned
9. **Personal Development Map v5** — Halfway update with reliability engineering growth evidence and refined intention for Modules 6–10

# Appendix C: Incident Simulation — Instructor Preparation Guide

**For instructors:** The Day 5 incident simulation requires significant preparation. This appendix provides guidance on creating a realistic, pedagogically effective simulation.

## Simulation Setup

- **Reference Application:** Use either (a) a dedicated reference app prepared by the instructor, or (b) a volunteer student's Module 3 app with their permission. Option (a) is safer because you can pre-test the failure chain.
- **Pre-test:** Run through the entire failure chain at least once before the live simulation. Ensure each failure is reproducible and recoverable.
- **Monitoring:** Ensure the reference app has the SLI dashboard from Day 1 visible on a projected screen. Students should see SLIs change in real-time as failures are introduced.
- **Communication channel:** Set up a shared Slack channel or use a physical whiteboard as the "incident channel" where teams post updates.

## Failure Chain Design

The failures should cascade — each new failure should interact with previous ones:

1. **09:05 — Database Degradation:** Introduce a slow query (add a sleep or remove an index). This is subtle — the app still works but latency creeps up. Students should notice p95 latency rising on the dashboard.
2. **09:20 — External API Failures:** Start returning intermittent 500 errors from a mocked external API. If students implemented a circuit breaker (Day 2), it should trigger. If not, the app hangs on every failed API call.
3. **09:40 — Memory Leak:** Start a background process that gradually allocates memory. P95 latency should climb steadily over 30 minutes. This tests whether students notice gradual degradation.
4. **10:00 — Traffic Spike:** Hit the already-degraded system with 10x normal traffic (use a load testing tool). This is the cascading moment — a slow database + high traffic + memory pressure = system meltdown. Students should see error rates spike on the dashboard.

## Debrief Talking Points

- Which teams detected the database degradation from the dashboard vs. from user reports? (monitoring value)
- Did anyone's circuit breaker fire correctly for the API failures? (integration pattern value)

- Did anyone notice the memory leak before the traffic spike? (gradual degradation is the hardest to detect)
- How did teams communicate during the incident? Was it organized or chaotic? (incident response maturity)
- What would have been different if this were a real production incident at 3 AM with no instructor help? (on-call reality)

---

*End of Module 5 Lesson Plan*